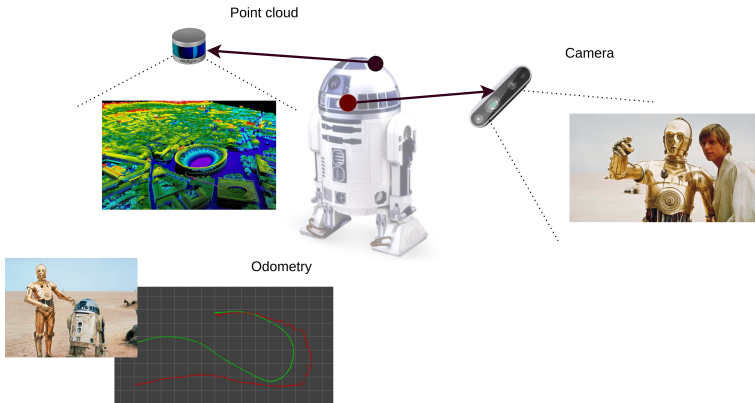


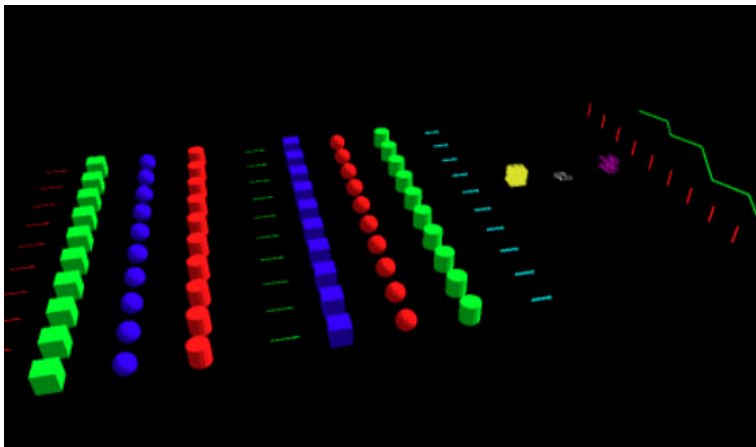
Introduction to ROS2: Basics, Motion, and Vision

ROS Visualization

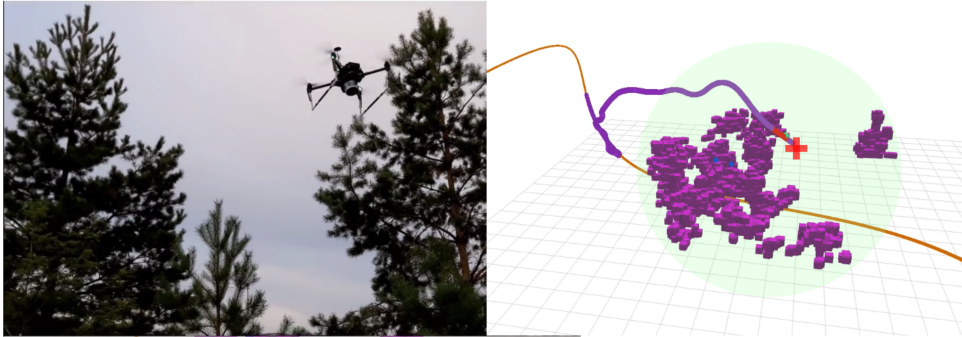


ROS Visualization

1 rviz <https://github.com/ros2/rviz>



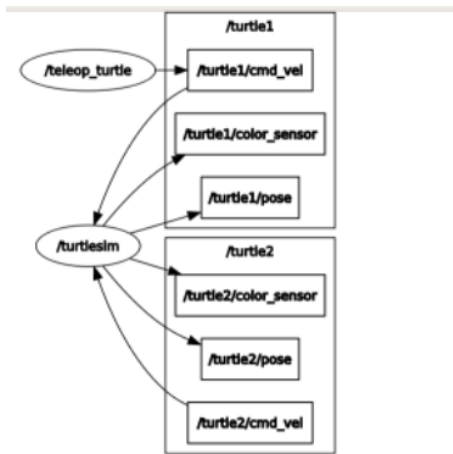
ROS Visualization



- Initial reference trajectory R — Prediction horizon P_{t_n} + Current pose
— Refined reference trajectory R_{t_n} ● Map update range $\textcircled{M}$

ROS Visualization

1 rqt <https://docs.ros.org/en/foxy/Concepts/About-RQt.html>



ROS Visualization with rqt

- 1 rqt can be used as a visualization tool as well as debugging tool
- 2 Scalar data can be easily visualized with rqt, however, non-scalar data such as images, pose, can also be visualized up to some extent in default setting
- 3 rqt consists of several sub-tools : **rqt**, **rqt_bag**, rqt_console, rqt_dep, **rqt_graph**, rqt_image_view, rqt_logger_level, rqt_plot and rqt_shell

ROS Visualization with rqt

- 1 Let's try to plot scalar data

to plot scalar data using rqt

```
ros2 launch ros2_visualize joy_hagen_sub.launch.py  
ros2 launch ros2_visualize joy_hagen_pub.launch.py
```

- 2 If the message consists of data that is not a scalar value, have to specify the fields separated by a comma,

to plot non-scalar data

```
/joy_hagen/imu/x
```

ROS Visualization with rqt

to visualize a single image

```
ros2 launch ros2_visualize joy_hagen_camera.launch.py
```


- 1 3D visualization is not possible with rqt, rviz, which is built with the help of OpenGL, can be used to visualize the 3D sensor data, e.g., stereo images, point clouds from camera
- 2 rviz can be launched in different ways:

to start rviz

```
rviz2
```

Visualization in ROS 2



- Visual feedback is essential for robotics development.
- RViz2 is the primary tool for visualizing:
 - TF frames
 - Marker messages
 - Sensor topics (e.g., point clouds, IMUs)
- We will use ROS 2 C++ to publish data for visualization.

Broadcasting TF Frames (C++)

Example: Broadcasting a transform from 'base_link' to 'camera_link'.

```
ros2 run tf2_ros static_transform_publisher 0.1 0 0 0 0 0 base_link camera_link
```

Arguments:

- 0.1 0 0 → translation (x, y, z) in meters
- 0 0 0 → rotation (roll, pitch, yaw) in radians
- base_link → parent frame ID
- camera_link → child frame ID

Result: A static transform from base_link to camera_link translated 0.1m along the x-axis, with no rotation.

Transforming Points and Frames in ROS 2 (C++)



Example Transform:

```
ros2 run tf2_ros static_transform_publisher \  
  0.1 0 0 0 0 0 base_link camera_link
```

Point in camera_link:

```
geometry_msgs::msg::PointStamped pt_camera;  
pt_camera.header.frame_id = "camera_link";  
pt_camera.point.x = 1.0;  
pt_camera.point.y = 0.0;  
pt_camera.point.z = 0.0;
```

Transforming Points and Frames in ROS 2 (C++)



Transform to base_link:

```
auto pt_base = tf_buffer.transform(pt_camera, "base_link");  
  
RCLCPP_INFO(node->get_logger(),  
    "Point in base_link: (%.2f, %.2f, %.2f)",  
    pt_base.point.x, pt_base.point.y, pt_base.point.z);
```

Expected Output:

- (1.1, 0.0, 0.0) shifted +0.1m in x-direction

Transforming Points and Frames in ROS 2 (C++)



Pose of camera_link in base_link:

```
auto tf = tf_buffer.lookupTransform("base_link", "camera_link", tf2::TimePointZero);

RCLCPP_INFO(node->get_logger(),
  "Translation: (%.2f, %.2f, %.2f)",
  tf.transform.translation.x,
  tf.transform.translation.y,
  tf.transform.translation.z);
```

Expected Pose:

- Translation: (0.1, 0.0, 0.0)
- Rotation (quaternion): (0, 0, 0, 1) no rotation

Publishing a Marker in C++

Example: Publish a red sphere marker.

```
visualization_msgs::msg::Marker marker;  
marker.header.frame_id = "map";  
marker.type = Marker::SPHERE;  
marker.action = Marker::ADD;  
marker.scale.x = 0.3;  
marker.scale.y = 0.3;  
marker.scale.z = 0.3;  
marker.color.r = 1.0;  
marker.color.a = 1.0;  
marker.pose.position.x = 1.0;  
marker.pose.orientation.w = 1.0;  
marker_pub->publish(marker);
```

Publishing a MarkerArray

```
visualization_msgs::msg::MarkerArray array;  
for (int i = 0; i < 5; ++i) {  
    Marker m;  
    m.header.frame_id = "map";  
    m.id = i;  
    m.type = Marker::CUBE;  
    m.pose.position.x = i * 0.5;  
    m.scale.x = m.scale.y = m.scale.z = 0.2;  
    m.color.g = 1.0;  
    m.color.a = 1.0;  
    array.markers.push_back(m);  
}  
marker_array_pub->publish(array);
```


Publishing Odometry

```
nav_msgs::msg::Odometry odom;  
odom.header.frame_id = "odom";  
odom.child_frame_id = "base_link";  
odom.pose.pose.position.x = 1.0;  
odom.twist.twist.linear.x = 0.5;  
odom_pub->publish(odom);
```

Publishing PointCloud2

Use OpenCV or raw loops to fill point data.

```
sensor_msgs::msg::PointCloud2 cloud;  
cloud.header.frame_id = "lidar_link";  
cloud.height = 1;  
cloud.width = points.size();  
cloud.fields = { /* x, y, z, intensity fields */ };  
cloud.is_dense = true;  
cloud.row_step = cloud.point_step * cloud.width;  
cloud.data.resize(cloud.row_step);  
memcpy(&cloud.data[0], &points[0], cloud.data.size());  
cloud_pub->publish(cloud);
```

- Set the correct fixed frame (`map`, `odom`, or `base_link`).
- Use the TF tree to debug transform issues.
- Important topics to visualize:
 - `/tf` and `/tf_static`
 - `/visualization_marker`, `/visualization_marker_array`
 - `/odom`, `/scan`, `/imu`, `/point_cloud`
- Use `rqt_graph` and `ros2 topic echo` to inspect topic flow and messages.

ROS Visualization: topics and frames

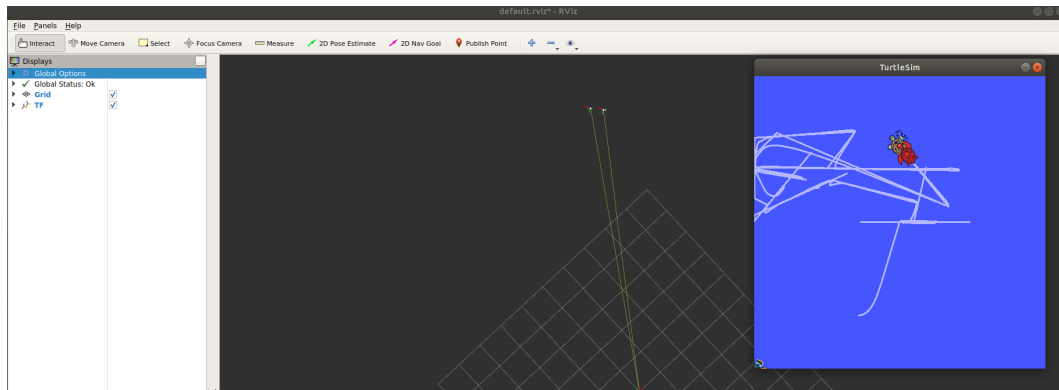


- 1 All the topics must have a `frame_id`, which defines the location of the sensor or device with respect to defined coordinate system
- 2 Usually, when we define the root of robot, it is named as the **base_link**, which is the common standard that everyone follows

to visualize frame of robots

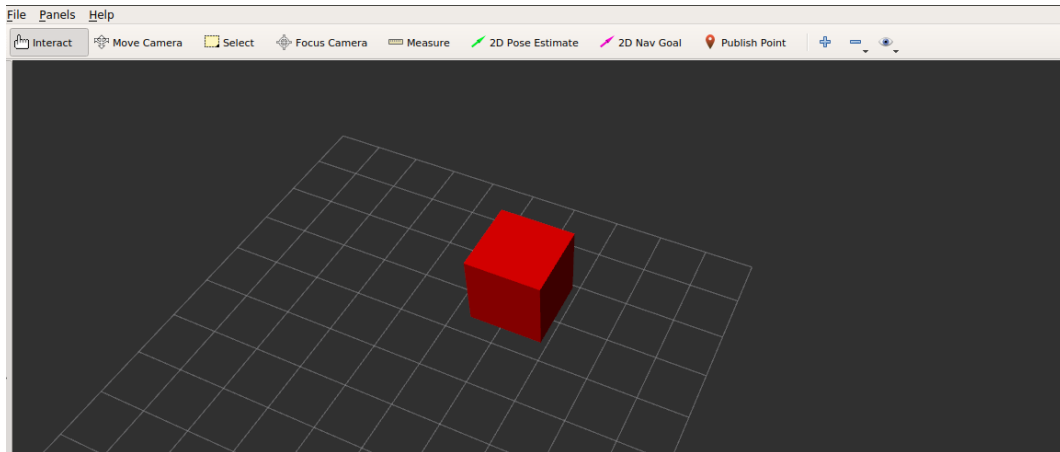
```
sudo apt-get install ros-humble-turtle-tf2-py ros-humble-tf2-tools ros-humble-tf-  
transformations  
ros2 run tf2_tools view_frames  
ros2 run tf2_ros tf2_echo turtle2 turtle1  
ros2 run rviz2 rviz2 -d $(ros2 pkg prefix --share turtle_tf2_py)/rviz/turtle_rviz.rviz
```

ROS Visualization: topics and frames

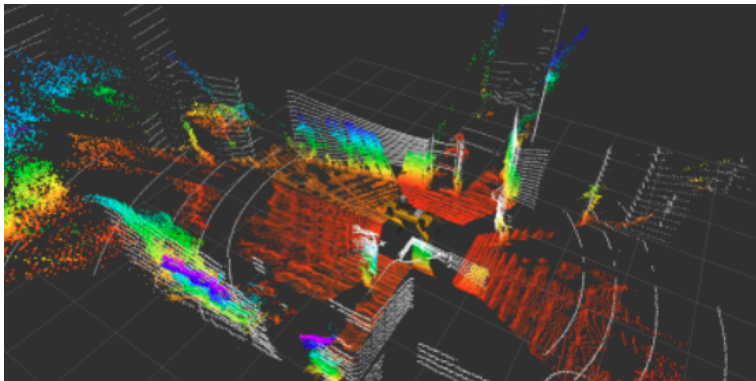


ROS Visualization: cube

Let's try to visualize a cube <http://wiki.ros.org/rviz/DisplayTypes/Marker>



ROS Visualization: point cloud



ROS Visualization: point cloud

Let's try to visualize a point cloud

